2026

Authors:

Ayodeji Olagoke

ADVICE DOCUMENT

OLAGOKE DEJI

Introduction

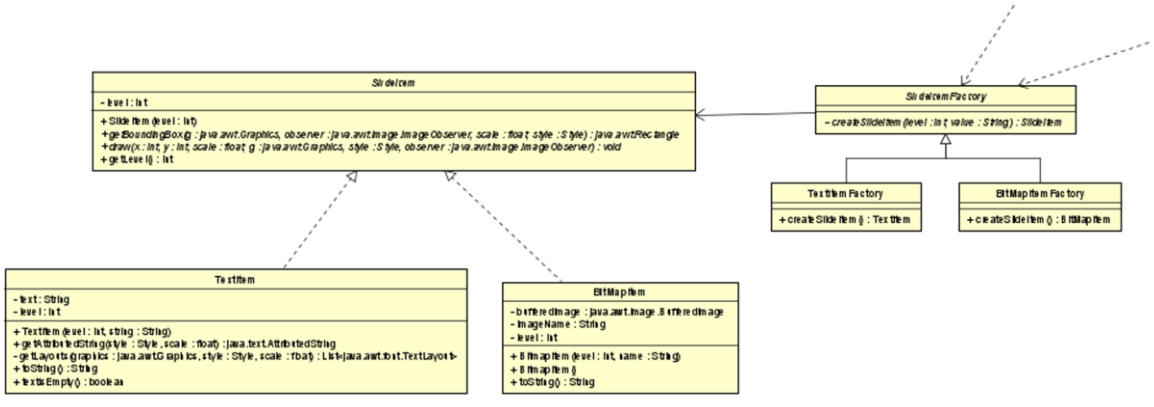
This report documents the analysis, refactoring, and redesign of the JabberPoint presentation application. The objective of this project was to improve the maintainability, readability, and extensibility of the existing codebase while working as the sole developer. The project began by analysing the original implementation to understand its architecture and identify areas for improvement. UML diagrams were created to visualise the system before any modifications were made, providing a clear understanding of the application's structure and behaviour. Based on this analysis, appropriate design patterns were implemented, followed by refactoring to improve code quality without changing the application's functionality.

Analysis of the Original System

Before implementing any changes, the original JabberPoint application was analysed using UML modelling techniques. An Activity Diagram was created to understand the program's execution flow, a Class Diagram was used to examine the relationships between classes, a Sequence Diagram illustrated how objects interact during execution, and a Use Case Diagram represented the functionality available to users. These diagrams established a baseline for understanding the existing architecture and identifying opportunities for improvement. Once the refactoring was complete, updated UML diagrams were produced to demonstrate how the architecture had evolved.

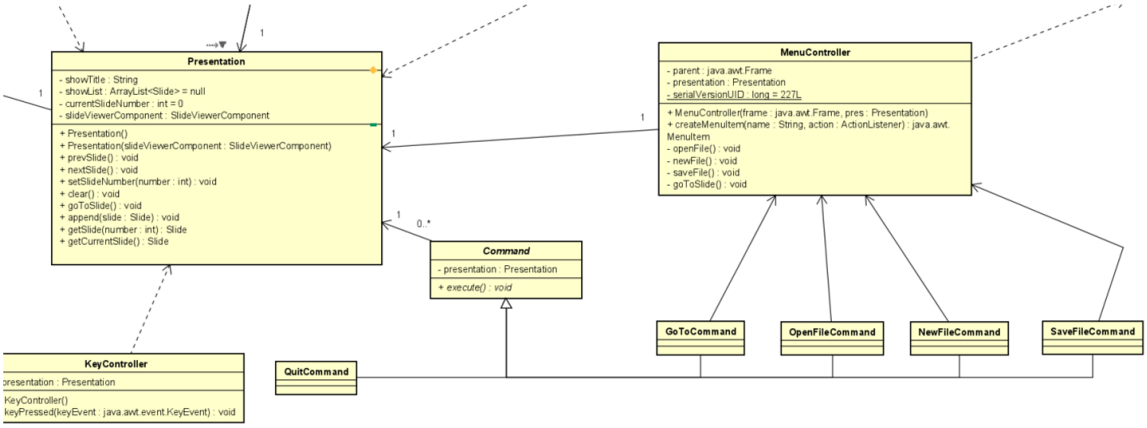
Factory Method Pattern

The Factory Method pattern was implemented to simplify the creation of SlideItem objects within the application. Instead of constructing objects directly throughout the codebase, dedicated factory classes were introduced to encapsulate the object creation process. This reduced duplicated construction logic, improved code organisation, and made the application easier to maintain. The Factory Method pattern also supports the Open/Closed Principle by allowing new SlideItem implementations to be added without modifying existing code. Alternative creational patterns such as Builder and Abstract Factory were considered; however, Builder was unnecessary because SlideItem objects require only a few construction parameters, while Abstract Factory introduced unnecessary complexity for creating individual objects rather than families of related objects.



Command Pattern

The Command pattern was introduced to encapsulate menu actions and user requests into individual command objects. Each command implements a common execute() method, allowing the menu controller to delegate actions rather than containing all application logic itself. This significantly reduced coupling between the user interface and the business logic while making the application easier to extend with new commands. Compared with the State and Mediator patterns, the Command pattern was the most suitable solution because it focuses specifically on encapsulating requests and separating the sender of an action from the object responsible for executing it.



Justification for choosing composite pattern as an additional design pattern

The Composite pattern is suitable for JabberPoint because the application's structure already follows a hierarchical model. A **Presentation** contains multiple **Slide** objects, while each **Slide** contains multiple **SlideItem** objects such as **TextItem** and **BitmapItem**. Although this hierarchy exists in the current implementation, it is not formalised through a common component interface.

The Slide.draw() method already demonstrates Composite behaviour by iterating through its child SlideItem objects and delegating the drawing operation to each item polymorphically. However, this uniform behaviour is not maintained throughout the system, as other parts of the code rely on explicit type checking.

1. Uniformity

The Composite pattern enables both individual objects (SlideItem) and composite objects (Slide, and optionally Presentation) to be accessed through a common interface. This allows client classes to perform operations without depending on concrete implementations.

Currently, methods such as Slide.getText() and XMLAccessor.saveFile() use instanceof checks to distinguish between TextItem and BitmapItem. Introducing a shared component interface would remove these conditional checks and provide a consistent way of interacting with all slide components.

2. Scalability

Composite supports the Open/Closed Principle by allowing new slide item types to be added without modifying existing client code.

For example, introducing a VideoItem or ShapeItem would only require implementing the component interface. Rendering, traversal and other operations would continue to function through polymorphism, avoiding changes to classes such as Slide and XMLAccessor.

3. Hierarchy Management

The pattern provides a clear representation of the application's part-whole relationships.

- Presentation → Slides
- Slide → SlideItems

This structure centralises responsibility for managing child components and allows operations such as drawing, exporting and layout calculation to be performed recursively across the hierarchy.

4. Extensibility

Formalising the hierarchy makes future enhancements easier to implement. Features such as grouped items, nested components, templates or additional slide content types can be incorporated without restructuring the existing design.

By moving type-specific behaviour into the leaf classes, responsibility is distributed appropriately and client code remains unchanged as the hierarchy evolves.

Summary

The Composite pattern is an appropriate structural solution because it reflects the existing hierarchy within JabberPoint while addressing current design limitations. It removes unnecessary instanceof checks, promotes uniform treatment of slides and slide items, improves maintainability, and supports future extension through polymorphism. Combined with the existing Factory Method and Command patterns, Composite provides a cleaner and more scalable architecture for managing presentation content.

Refactoring

The original JabberPoint codebase contained several maintainability issues, including poor code organisation, overly large classes, unclear naming conventions, duplicated code, and comments that added little value. The refactoring process focused on improving readability and maintainability while preserving the application's existing functionality. Classes and methods were reorganised to follow the Single Responsibility Principle, variable and method names were made more descriptive, duplicated logic was removed, and reusable functionality was extracted into utility classes. The project structure was improved to separate concerns more effectively, making the codebase easier to understand and extend. Unit tests were also introduced to verify that functionality remained correct throughout the refactoring process.

Development Process (DTAP)

A structured Git workflow was adopted to support the development process. Although this project was completed individually, a branching strategy was used to simulate a professional development environment. The **main** branch contained stable production-ready code, the **test** branch was responsible for running automated tests and code coverage, and the **dev** branch was used for ongoing development and feature implementation. Continuous Integration pipelines were configured to

compile the project, execute unit tests, generate code coverage reports, and enforce consistent coding standards through automated formatting and linting. This workflow ensured that only tested and properly formatted code was merged into the production branch.

Conclusion

The refactoring of the JabberPoint application successfully improved the overall architecture, maintainability, and extensibility of the software. Through the implementation of the Factory Method, Command, and possibly the Composite design patterns, the application became more modular and better aligned with object-oriented design principles.